

Lab-1 Assignment

Procedure

Accept the assignment

On URCourses you will find a link to a GitHub classroom assignment. Accept this assignment and make sure you select your User ID from the list provided so you submit under your username. You must accept this assignment on your assigned lab day.

Creation of a new Project

We are going to create a project based on a simulated target. This means that we will not need our evaluation board at this time. Our ARM core will be simulated. It is even possible to simulate peripherals in some cases.

1. Start the Keil UVision5 tool from the Desktop icon or from the Start Menu.
2. Select Project → New UVision Project...
3. In the Create New Project dialog, select one of your network drives to store the project in.
4. Open the STMicroelectronics section → STM32F1 Series → STM32F103 → STM32F103RB.
5. Say No if prompted about STM32 Startup Code.
6. Add the project and project files to your repository.

Adding a source file

1. Select File → New. A text window will open and you can begin entering your program. Begin with a comment line similar to the following:
`;First Program – Your Name, Date`
2. Select File → Save. In the dialog, give the file a meaningful name and a `.s` extension. Make sure it gets saved in the same folder as your project.
3. Right click on the Source Group 1 folder in your project navigator. Select Add Files to Group... In the Files of Type drop down box, choose Asm Source file (.s). Now select your .s file and click Add.
4. Close the dialog.
5. Add your source file to your repository.

Skeleton Code Listing

On our course website you can find a sample file to use. You do have to be careful with your code alignment.

Writing your source code

1. From the information given, the sample code, and the CortexM3UserGuide.
2. Implement a program that will use 3 general purpose registers to perform storage and addition of hexadecimal numbers. Regardless of the actual registers, we'll refer to them as Rx, Ry, and Rz.

- Load the number `0x00000001` into Ry, `0x00000002` into Rz
 - Add Ry and Rz without affecting the condition codes, and store the result in Rx
 - Load `0xFFFFFFFF` into Rz
 - Add Ry and Rz affecting the condition codes, and store the result in Rx. Note what the condition flags are
 - Load `0x2` into Ry
 - Add Ry and Rz affecting the condition codes, and store the result in Rx. Note what the condition flags are
 - Load `0x7FFFFFFF` into Ry and `0x7FFFFFFF` into Rz
 - Add Ry and Rz affecting the condition codes, and store the result in Rx. Note what the condition flags are
 - The program needs to run in a loop, performing all these steps again
3. Once you've written the program, save it and then select Project → Build target. Use the output in the Build Output window to reference and correct any errors.
 4. Go to the Download and Debug section and test your program.

Submission

Make sure you have committed your latest solution and pushed it to your repository for grading.